≡

INTRODUCTORY COURSE · MODULE 01

Machine Learning Fundamentals

From data and algorithms to intelligent predictions — how machines find patterns in examples, and turn those patterns into decisions.

Supervised Learning · Unsupervised Learning · scikit-learn · 40 slides

PART 01 · WHAT IS MACHINE LEARNING

The Big Question

"How can a computer learn to make predictions without being explicitly programmed for every possible situation?"

Spam detection
Sort incoming email as spam or not, without a hand-written rule for every trick spammers invent.

Movie recommendations
Suggest what you'll want to watch next, based on patterns in what you and similar viewers watched.

House price prediction
Estimate a fair price from size, location, and condition — without a fixed pricing formula.

Fraud detection
Flag a suspicious transaction in milliseconds, from patterns across millions of past transactions.

Customer churn prediction
Predict which customers are about to leave, before they cancel.

Image classification
Recognize what's in a photo — a cat, a tumor, a road sign — without describing every possible pixel pattern.

Ask yourself: what exactly does "learning" mean for a computer? Keep that question open — we'll answer it precisely by the end of this module.

PART 02 · A DIFFERENT WAY TO PROGRAM

Traditional Programming vs. Machine Learning

The same inputs, rearranged — with the roles of "rules" and "output" swapped.

TRADITIONAL PROGRAMMING

Data + Rules → Output

A human writes the rules. The computer just executes them.

MACHINE LEARNING

Data + Expected Outputs → Learning Algorithm → Model

The computer figures out the rules from examples.

New Data + Model → Prediction

Why not just write the rules by hand? Consider spam: "If email contains 'FREE' → spam" catches one trick and misses the next thousand. Real language has too many patterns, exceptions, and adversaries for any fixed rule list to scale.

PART 03 · THE MENTAL MODEL

The Core Idea

Machine learning finds useful patterns in data, and uses those patterns to make predictions or decisions on new data.

Data → Patterns → Model → Prediction

The model is not the data. Once trained, a model doesn't store your dataset — it stores a compact, learned representation of the patterns inside it.

A house-price model doesn't remember every house it saw. It learns something like "price tends to rise with size, in this kind of way" — and applies that relationship to houses it has never seen.

PART 04 · THE RAW MATERIAL

What Is Data?

A realistic example: a student performance dataset.

HOURS STUDIED	ATTENDANCE	PREVIOUS SCORE	FINAL SCORE
2	70%	55	58
4	80%	62	68
6	90%	75	82
8	95%	82	91

Rows and columns
Rows are observations — one student, one exam cycle. **Columns** are variables — one measured quantity per student.

Features and target
Columns used as inputs are **features**. The column we want to predict is the **target**. In notation: X → features, y → target.

PART 05 · X AND Y

Features and Targets

The same X → y pattern, across three different problems.

HOUSE PRICE PREDICTION
FEATURES (X)
- Area
- Bedrooms
- Location
- Age

TARGET (y)
Price

SPAM DETECTION
FEATURES (X)
- Word frequency
- Sender information
- Email length
- Links

TARGET (y)
Spam / Not Spam

CUSTOMER CHURN
FEATURES (X)
- Usage
- Subscription type
- Login frequency
- Support tickets

TARGET (y)
Churn / No Churn

YOUR TURN

A streaming service wants to predict the star rating a viewer will give a movie before they watch it. What would be the features, and what would be the target?

Reveal answer

PART 06 · THREE PARADIGMS

Types of Machine Learning

This course focuses on the first two. The third is a distinct paradigm worth knowing about.

Supervised Learning
Data + known answers. The algorithm learns a mapping from inputs to correct outputs.

Unsupervised Learning
Data + no known answers. The algorithm discovers structure on its own.

MENTIONED BRIEFLY
Reinforcement Learning
Learning through actions, rewards, and penalties — an agent improves by interacting with an environment. A separate paradigm from the two above; not covered in depth here.

PART 07 · LEARNING FROM EXAMPLES

Supervised Learning

Like a student checking answers against a key — except the algorithm sees thousands of worked examples, and generalizes the pattern connecting question to answer.

What it receives
Input → Correct Output
Repeated over every row of the training data.

What it learns
f(X) → y
A function that maps features to a target — accurate enough to use on data it has never seen.

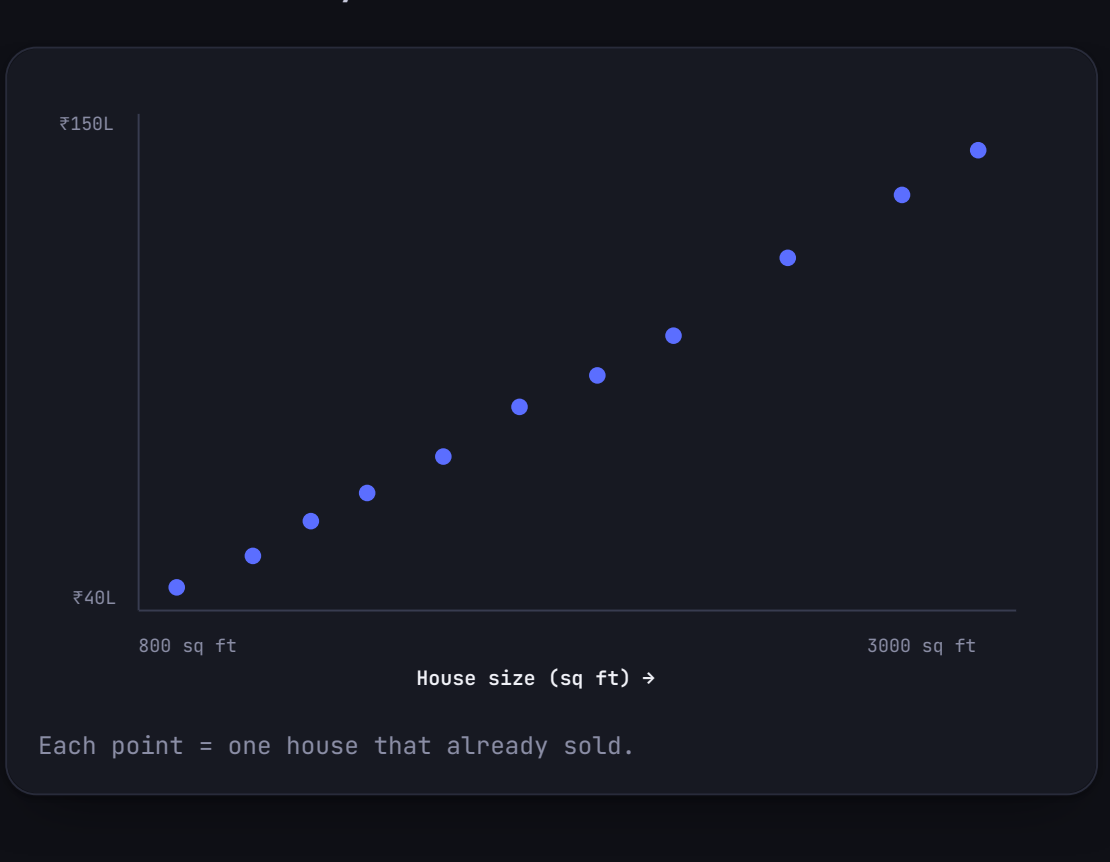
Regression
Predict a **number**. e.g. house price, exam score, temperature.

Classification
Predict a **category**. e.g. spam / not spam, cat / dog.

PART 08 · PREDICTING A NUMBER

Regression

House price prediction: plot house size against price for homes that already sold.



Each point = one house that already sold.

The data approximately follows a trend: bigger houses tend to cost more.

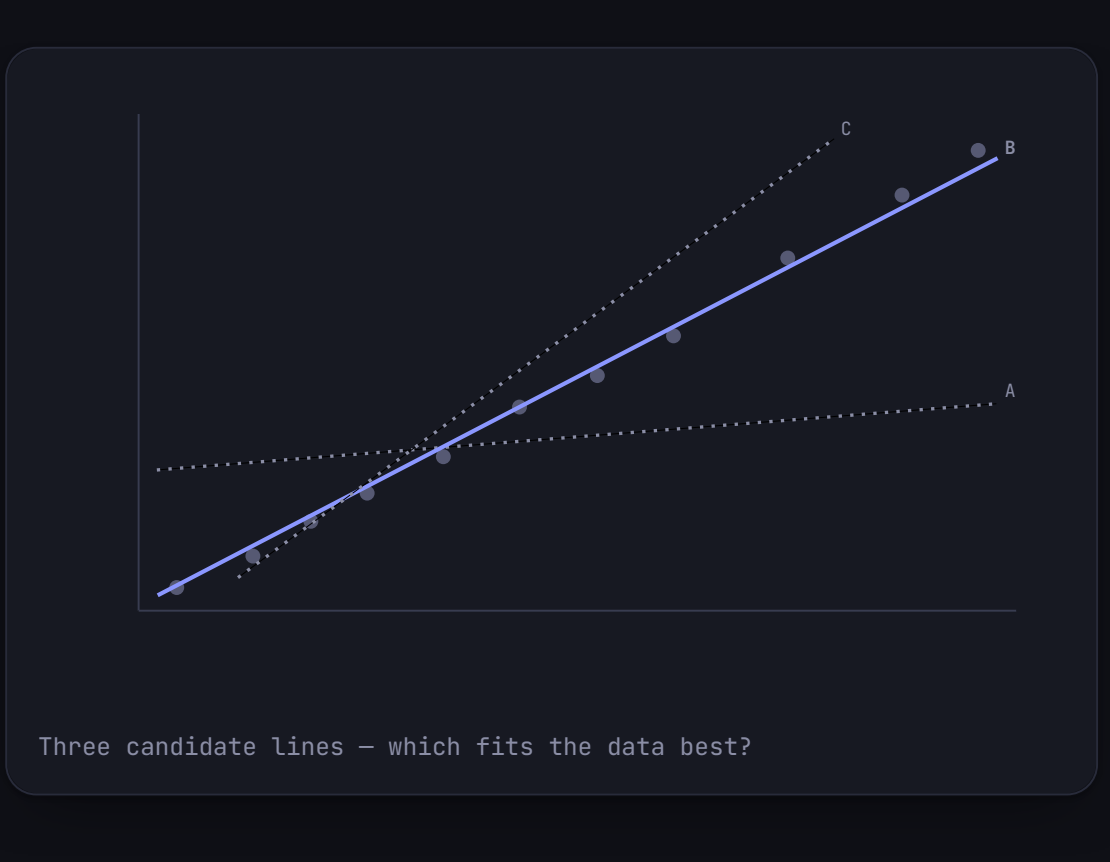
The question: if we know the size of a new house, can we estimate its price from this trend?

That's exactly what **linear regression** does.

PART 09 · FITTING A LINE

Linear Regression

The visual idea: find a line that best represents the relationship between X and y.



Three candidate lines — which fits the data best?

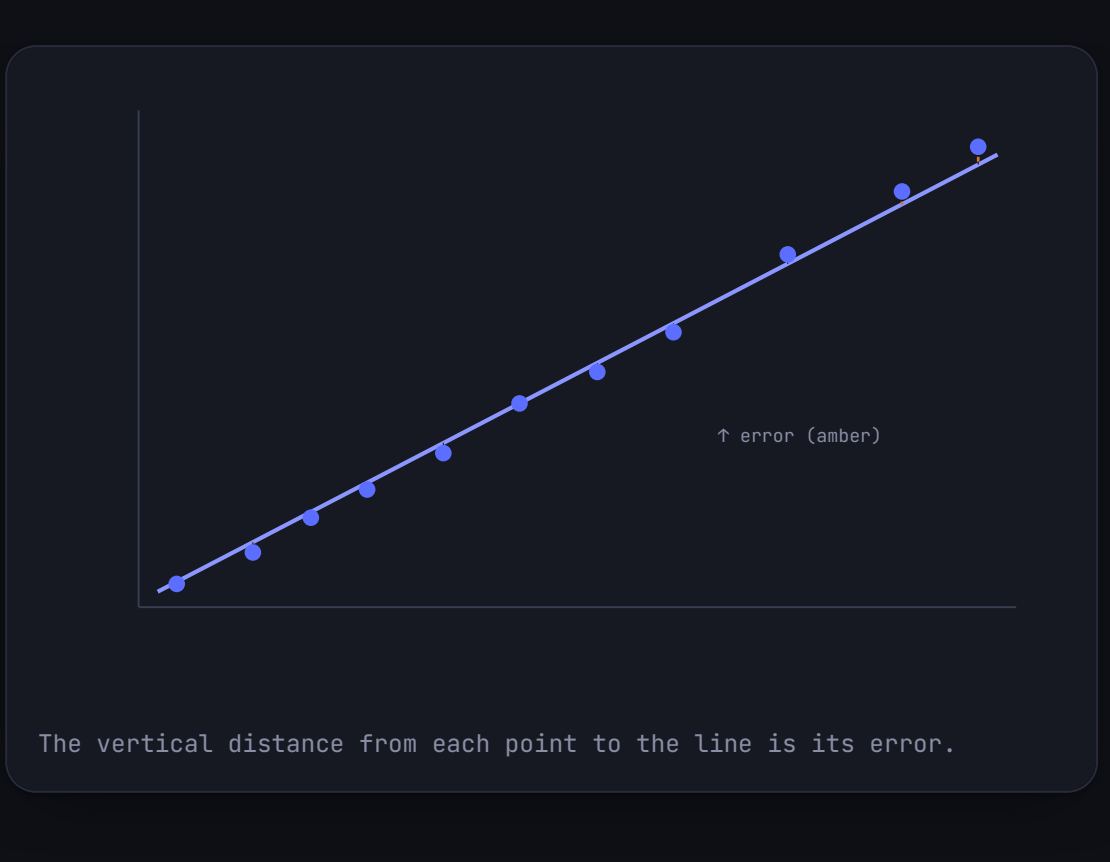
$$y = mx + b$$

- **x** — input (house size)
- **y** — prediction (price)
- **m** — slope (price increase per sq ft)
- **b** — intercept (baseline price)

Line B looks closest by eye — but "looks closest" isn't a rigorous answer. We need a way to **measure** how good a line is.

PART 10 · MEASURING ERROR

What Does "Best" Mean?



The vertical distance from each point to the line is its error.

Error = Actual - Predicted

To turn many individual errors into one score for the whole line, we use **Mean Squared Error**:

$$MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$$

- Calculate the error for each point
- Square it (so positive and negative errors don't cancel out, and big misses count more)
- Average the errors across all points

The model wants to **minimize** this number. The line with the lowest MSE is "best."

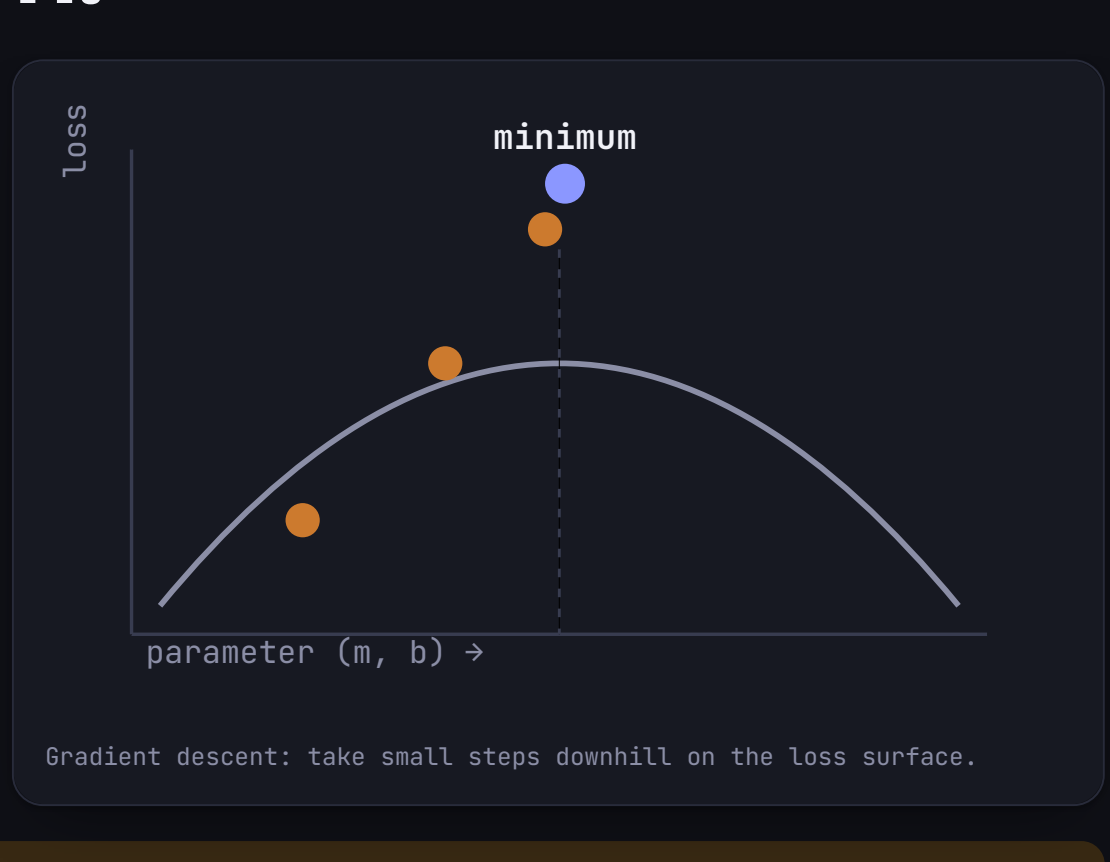
PART 11 · TRAINING

How Does the Model Learn?

Start with parameters → Make predictions → Calculate loss (MSE) → Adjust parameters

↻ repeat

Each lap through this loop is one step of **optimization** — nudging m and b to reduce the loss.



Gradient descent: take small steps downhill on the loss surface.

This is **optimization**: search for parameter values that minimize loss. The algorithm that does the searching here is called **gradient descent** — no calculus needed to grasp it: just picture rolling a ball downhill until it settles at the bottom.

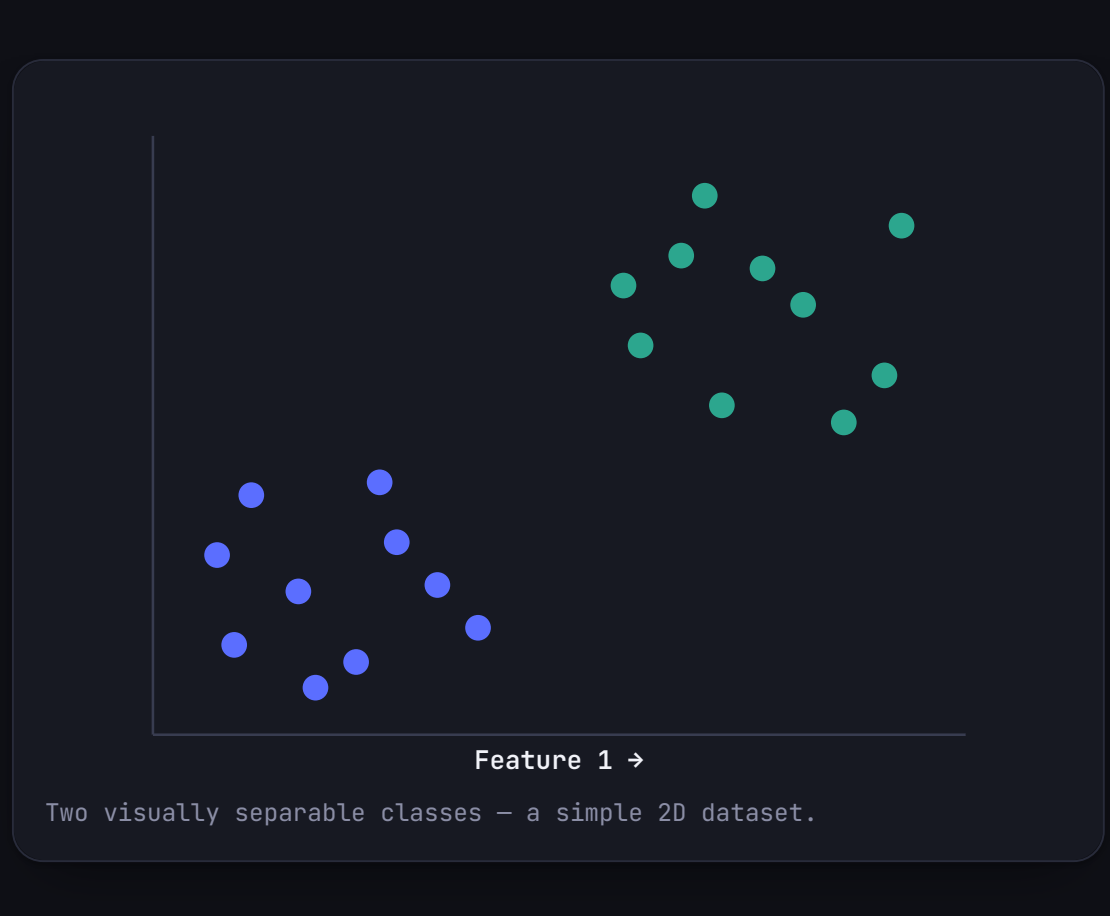
PART 12 · PREDICTING A CATEGORY

Classification

Change the problem. Instead of predicting `₹65 lakh`, we predict `Spam / Not Spam` or `Cat / Dog`.

Classification predicts **discrete categories** instead of continuous numbers. The output is a label, not a quantity you could average or subtract.

● Class A ● Class B

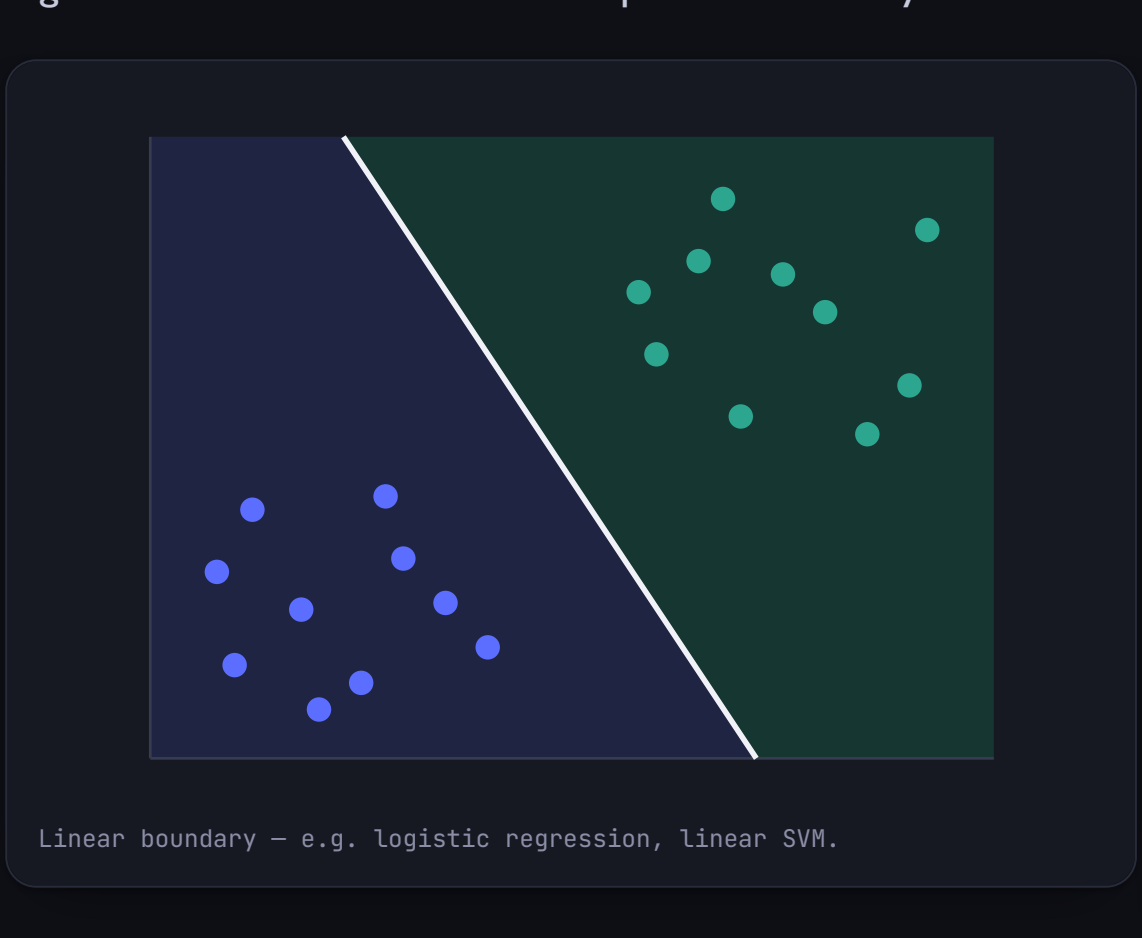


Two visually separable classes — a simple 2D dataset.

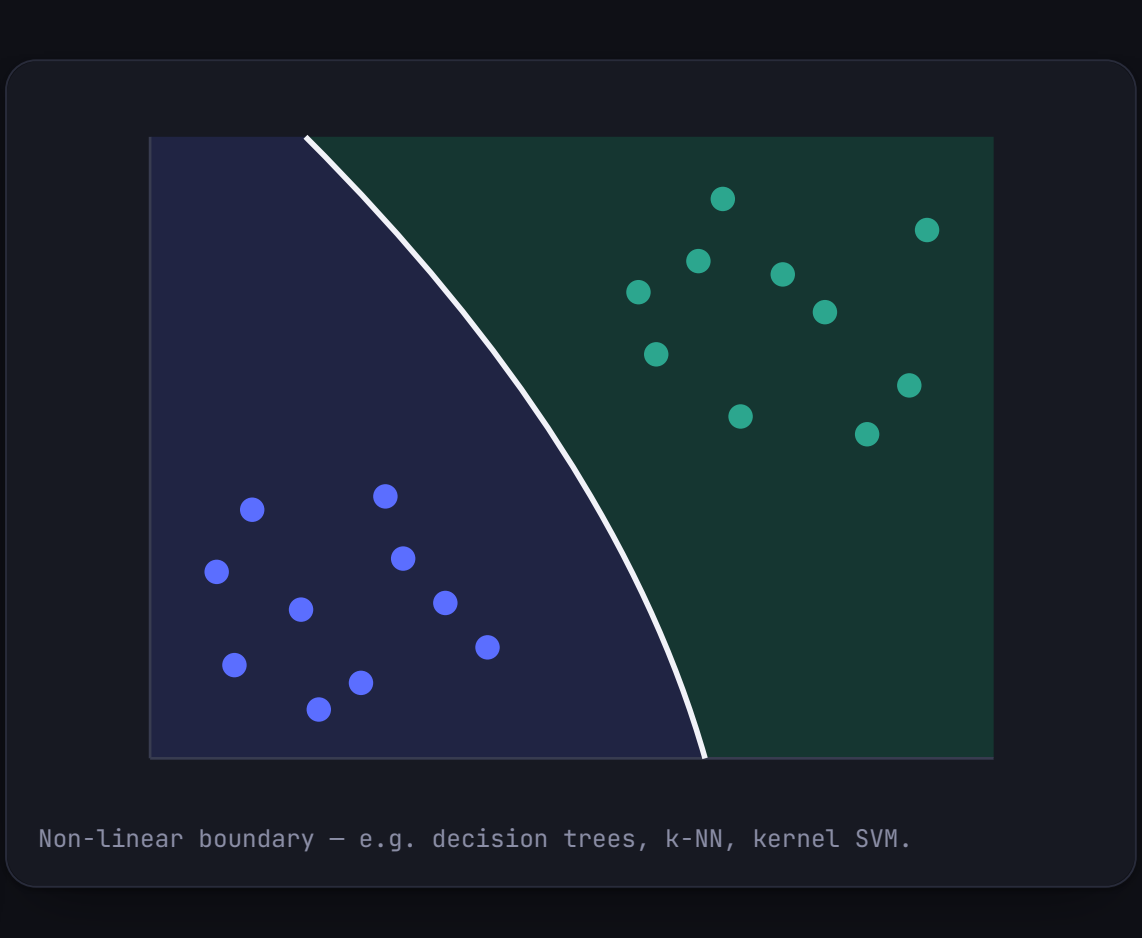
PART 13 · SEPARATING THE CLASSES

Decision Boundaries

A classifier learns a boundary that separates different classes. Different algorithms can learn different shapes of boundary.



Linear boundary — e.g. logistic regression, linear SVM.



Non-linear boundary — e.g. decision trees, k-NN, kernel SVM.

PART 14 · THE CLASSICAL TOOLKIT

Common Classical ML Algorithms

Not taught in depth here — just what kind of problem each one is built to solve.

- **Linear Regression** — fits a straight-line relationship
- **Polynomial Regression** — fits curved relationships

- **Logistic Regression** — linear boundary, probability output
- **Decision Trees** — splits data by yes/no questions
- **Random Forest** — many trees voting together
- **k-Nearest Neighbors** — classifies by closest examples
- **Support Vector Machines** — maximizes the margin between classes

- **K-Means** — groups data into K clusters
- **Hierarchical Clustering** — builds a tree of nested groups
- **PCA** — compresses features while keeping the signal

PART 15 · DID IT ACTUALLY LEARN?

Training vs. Testing

"How do we know whether our model actually learned — or just memorized?"

Training Data → Learn   Test Data → Evaluate

Analogy: practice questions ≠ the final examination. A model must perform well on data it was never trained on — that's the only honest test of whether it learned the underlying pattern, rather than the specific examples.

PART 16 · THE REALISTIC WORKFLOW

Train / Validation / Test

One dataset, split three ways, each with a distinct job.

Training — 60% | Validation — 20% | Test — 20%

Training
Used to learn parameters — where model.fit() does its work.

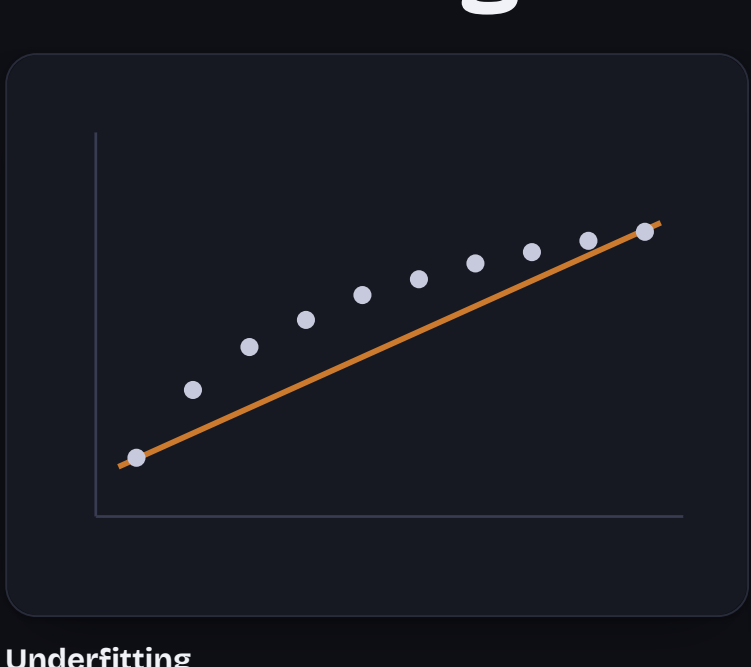
Validation
Used to make modeling decisions — comparing algorithms, tuning settings.

Test
Used once, at the end, for a final unbiased evaluation.

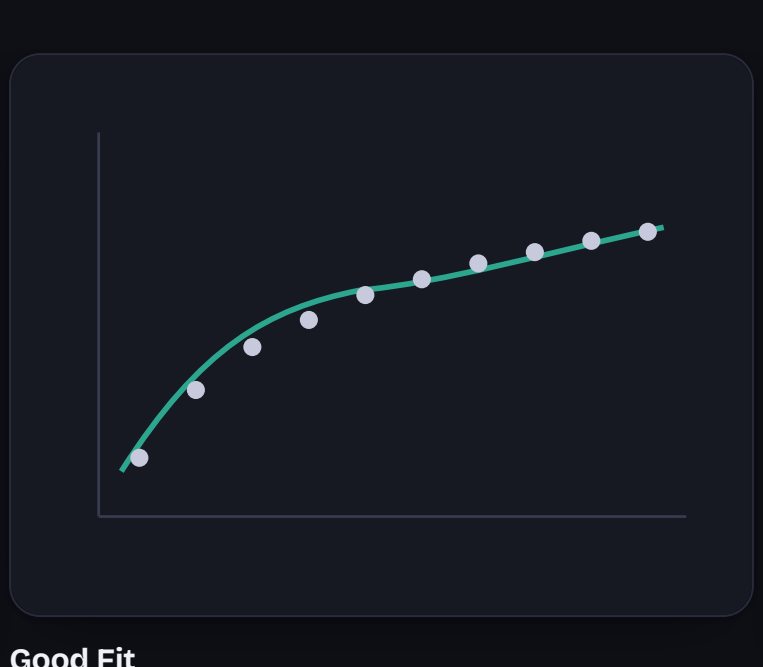
Why not just keep peeking at the test set? Every time you adjust the model based on test results, you're indirectly fitting to the test set — and its score stops being an honest estimate of real-world performance.

PART 17 · TOO SIMPLE, TOO COMPLEX, JUST RIGHT

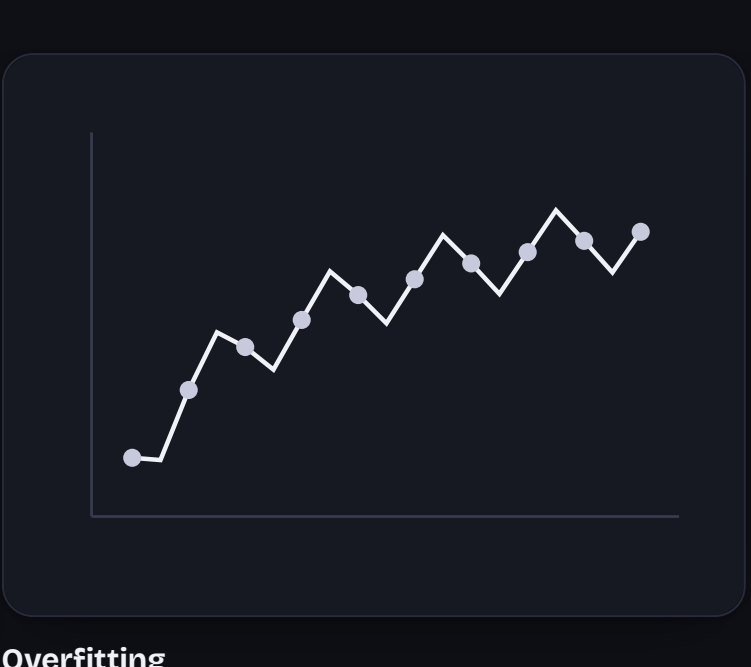
Overfitting



Underfitting
Too simple. Misses the real curvature in the data.



Good Fit
Captures the meaningful pattern, ignores the noise.



Overfitting
Learns the noise. Memorizes training examples instead of the trend.

PART 18 · THE GOAL

Generalization

A good ML model should perform well on data it has never seen before.

`Training performance` ≠ `Real-world performance`

Generalization is the direct opposite of overfitting: a model generalizes well when the patterns it learned hold up outside the training set — on the data it will actually face in production.

PART 19 — I · SCORING A CLASSIFIER

Classification Metrics: The Confusion Matrix

Fraud detection: every prediction falls into one of four boxes.

	PREDICTED: FRAUD	PREDICTED: NOT FRAUD
ACTUAL: FRAUD	TP True Positive	FN False Negative
ACTUAL: NOT FRAUD	FP False Positive	TN True Negative

True Positive — flagged fraud that really was fraud.

True Negative — passed a transaction that really was legitimate.

False Positive — flagged a legitimate transaction as fraud.

False Negative — missed actual fraud.

PART 19 — II · BEYOND ACCURACY

Precision, Recall & the Accuracy Trap

Accuracy
`(TP+TN)/Total`

Precision
`TP/(TP+FP)`

Recall
`TP/(TP+FN)`

F1 Score
`2PR/(P+R)`

WHY ACCURACY ALONE CAN MISLEAD

99,000 of 100,000 transactions are legitimate; only 1,000 are fraud. A model that predicts "Not Fraud" for every single transaction is right 99% of the time.

Is that a good model?

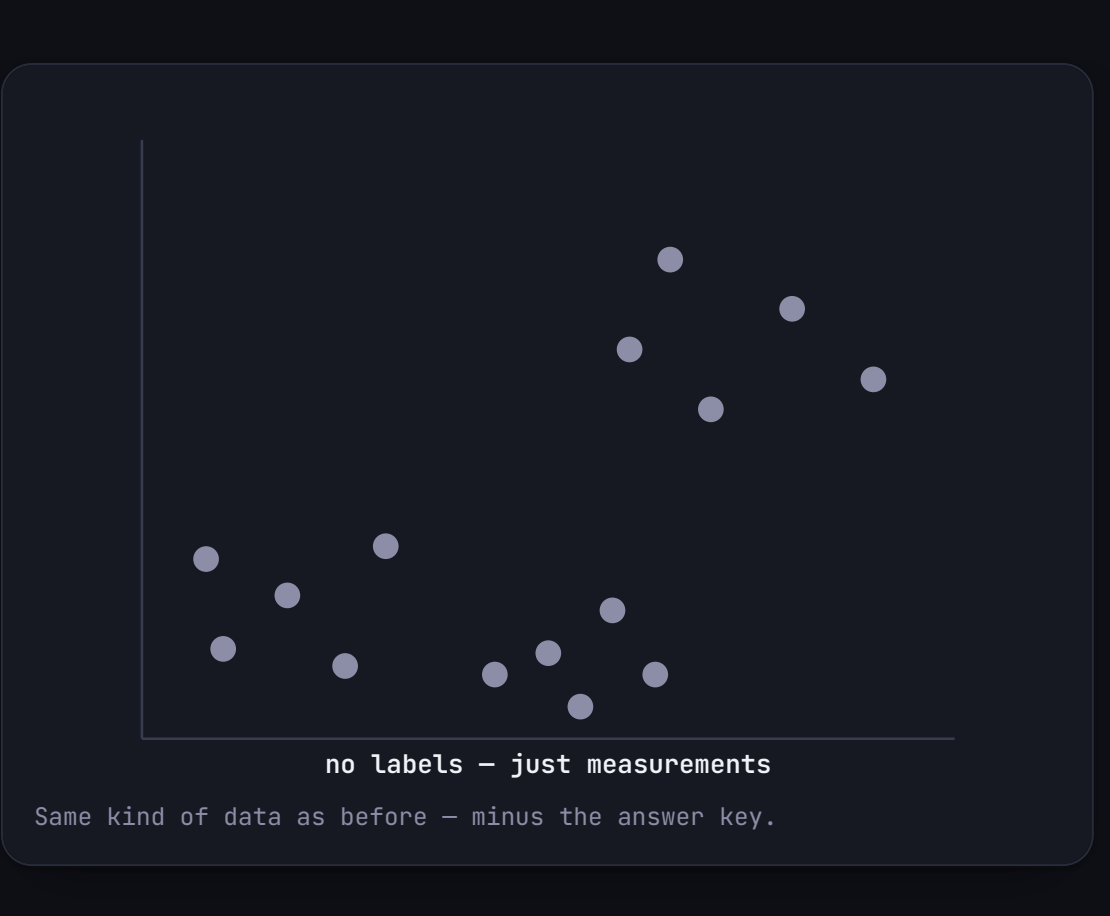
PART 20 · NO ANSWER KEY

Unsupervised Learning

Remove the labels. Give the algorithm customer data — but don't tell it which customers belong together.

> **The question:** can the algorithm discover structure by itself, with no correct answers to learn from?

This is **clustering** — grouping similar examples together based on patterns in the features alone.

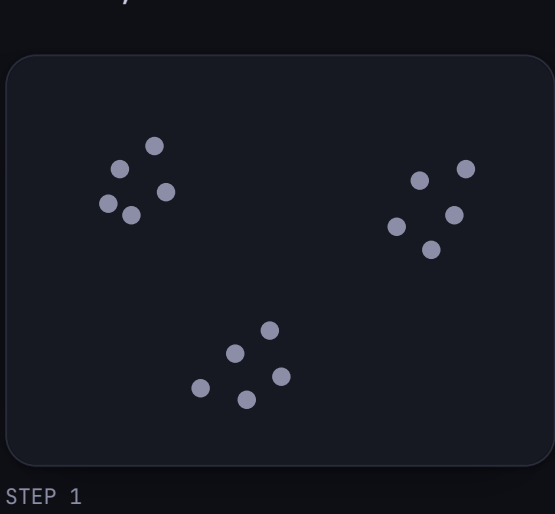


Same kind of data as before — minus the answer key.

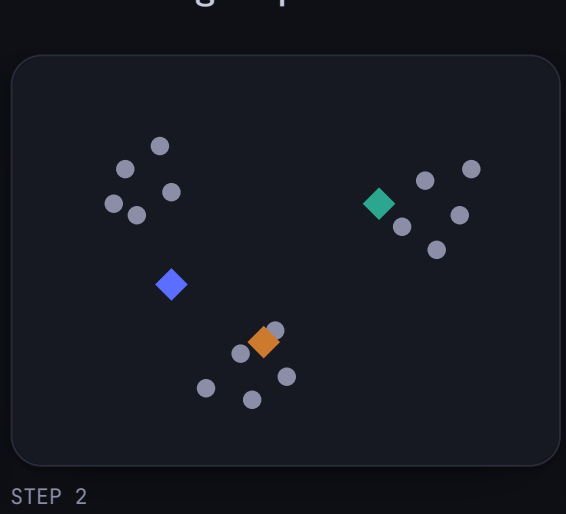
PART 21 · GROUPING BY DISTANCE

K-Means

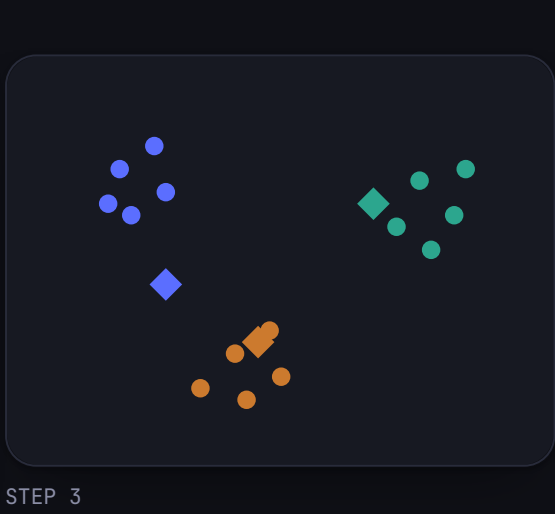
The intuition, before the equations: repeatedly assign points to their nearest center, then move the center to match its group.



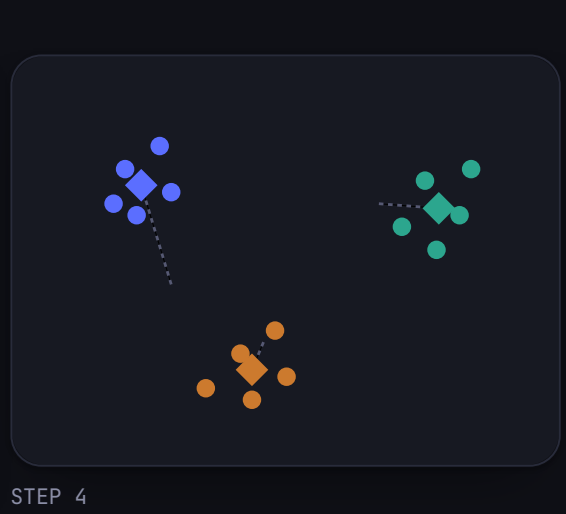
STEP 1
Choose K = 3



STEP 2
Initialize centroids



STEP 3
Assign points



STEP 4
Recalculate & repeat

Repeat steps 3–4 until the centroids stop moving. No labels were ever provided — the structure emerged from distance alone.

PART 22 · BETTER INPUTS, BETTER MODELS

Feature Engineering

Machine learning isn't "throw data into an algorithm." What you feed the model matters as much as the algorithm itself.

`Raw Data` → `Useful Features` → `Model` → `Prediction`

From a single date
`Date of Birth` → `Age`

From a single timestamp
`Transaction Timestamp` → `Hour` `Day` `Weekend?` `Month`

A raw timestamp is nearly meaningless to a model. Hour-of-day and weekend-or-not can be exactly the signal that predicts fraud or demand.

PART 23 · WHAT THE MODEL ACTUALLY LEARNS FROM

Data Quality

Machine learning models learn from the data we give them — nothing more, nothing less.

Missing values
Gaps that force assumptions or exclusions.

Outliers
Extreme values that can dominate the fit.

Incorrect labels
Wrong answers the model treats as ground truth.

Duplicate data
Silently over-weights repeated examples.

Biased data
Skewed sampling bakes in unfair patterns.

Data leakage
Future or answer information sneaks into training.

> **Garbage in → garbage out.** But data problems are often far more subtle than "bad data" — a perfectly clean-looking dataset can still be biased, leaky, or unrepresentative.

PART 24 · THE FEATURE THAT CHEATS

Data Leakage

SCENARIO

Goal: predict whether a customer will cancel their subscription. A feature in the dataset is called "Cancellation date."

Should we use this feature?

> **Data leakage** occurs when information that wouldn't be available at prediction time accidentally enters the training data.

It's one of the most common — and most dangerous — mistakes in applied ML, because a leaky model can look excellent during evaluation and fail immediately in the real world.

PART 25 · FROM THEORY TO CODE

Build Your First ML Model

Python and scikit-learn. Focus on what's happening mathematically and conceptually — not the syntax.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X = df[["area", "bedrooms"]]   # features
y = df["price"]                  # target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

X / y split
Separates inputs from the answer the model must predict.

train_test_split
Holds out 20% of the data the model will never train on.

fit / predict
fit() runs the training loop from Part 11; predict() applies the learned line to new houses.

PART 26 · SCORING THE MODEL

Evaluate the Model

```python
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_test, predictions)
print(mae)
```

MAE = average absolute prediction error, in the same units as the target.

INTERPRET, DON'T JUST CALCULATE

If the model's MAE is ₹3 lakh, what does that actually mean?

Reveal interpretation

PART 27 · THE FULL PICTURE

The Complete ML Workflow

1. **Define the Problem**
   What exactly are we predicting?
2. **Collect Data**
   Gather examples with features and outcomes
3. **Explore Data**
   Look at distributions, relationships, oddities
4. **Clean Data**
   Handle missing values, outliers, errors
5. **Engineer Features**
   Turn raw fields into useful signals
6. **Split Data**
   Train / validation / test
7. **Train Model**
   Fit parameters to the training set
8. **Evaluate**
   Score on held-out data
9. **Tune / Improve**
   Adjust features, model, or settings
10. **Deploy**
    Put the model into production
11. **Monitor**
    Watch performance as real-world data arrives

> **Machine learning is a workflow** — not just calling `model.fit()`.

PART 28 · BEYOND THE NOTEBOOK

A Real-World ML System

Production ML involves far more than training a model once.

`User`
↓
`Application`
↓
`Feature Processing`
↓
`ML Model`
↓
`Prediction`
↓
`Business Decision`
↓
`Feedback / New Data`

> **The loop closes:** feedback and new data flow back in, driving *Model Improvement* — the system keeps learning after deployment, not just before it.

A notebook produces a model. A production system wraps that model in infrastructure: feature pipelines, monitoring, retraining, and a feedback loop back to the start.

PART 29 · TWO TOOLBOXES

Classical ML vs. Deep Learning

Classical ML
- › Often works well with structured / tabular data
- › Often requires manual feature engineering
- › Many algorithms are interpretable
- › Works well with smaller datasets

Deep Learning
- › Built on neural networks
- › Often learns representations automatically
- › Excels with images, audio, text, and other complex data
- › Often requires more data and compute

> **Neither is "better."** Different problems require different tools — a tabular dataset with 2,000 rows rarely benefits from a deep network, and raw pixels rarely benefit from hand-built features alone.

PART 30 · WHERE THIS SHOWS UP

ML in the Real World

Fraud detection
`Transaction → Fraud probability → Block or flag`

Recommendations
`History → Relevance score → Show top-N items`

Credit risk
`Financial history → Default probability → Approve / price loan`

Demand forecasting
`Past sales, season → Expected demand → Set inventory levels`

Predictive maintenance
`Sensor readings → Failure probability → Schedule maintenance`

Search ranking
`Query + signals → Relevance score → Order results`

Customer churn
`Usage, support history → Churn probability → Retention offer`

Medical diagnosis assist
`Scans, patient data → Condition likelihood → Flag for clinician`

Spam detection
`Content, metadata → Spam probability → Filter to spam folder`

Image classification
`Pixel data → Predicted label → Tag, sort, or route`

PART 31 · BEFORE YOU BUILD A MODEL

Think Like an ML Engineer

- What exactly are we predicting?
- What data do we have?
- What information is available at prediction time?
- What features might matter?
- What does success mean?
- What metric should we use?
- How will we detect overfitting?
- How will the model behave on unseen data?
- What happens when the data changes?

PART 32 · PROJECT

Hands-On Challenge: Predict Student Performance

COLUMN	ROLE
Hours studied	Feature

- Explore the data
- Visualize relationships

Attendance	Feature
Previous score	Feature
Assignments completed	Feature
Sleep hours	Feature
Final exam score	Target

- Select features
- Split the dataset
- Train Linear Regression
- Make predictions
- Calculate MAE and MSE
- Plot actual vs. predicted values
- Identify possible outliers
- Explain whether the model generalizes well

PART 33 — I · QUICK CHECK

Regression, Classification, or Clustering?

QUESTION 1

Predict house price. Regression or classification?

Reveal

QUESTION 2

Predict whether a customer will churn. Regression or classification?

Reveal

QUESTION 3

Group customers with no labels provided. Which learning type?

Reveal

PART 33 — II · REASON IT OUT

What's Going Wrong?

QUESTION 4

A model achieves 99% training accuracy and 65% test accuracy. What might be happening?

Reveal answer

PART 33 — III · REASON IT OUT

Is 99% Accuracy Good?

QUESTION 5

A dataset contains 99,000 non-fraud transactions and 1,000 fraud transactions. A model achieves 99% accuracy. Is that necessarily a good model?

Reveal answer

PART 34 · TYING IT TOGETHER

The Final Mental Model

DATA What examples do we have?

↓

FEATURES What information describes each example?

↓

MODEL What pattern can we learn?

↓

LOSS How wrong are our predictions?

↓

OPTIMIZATION How can we improve?

↓

EVALUATION Does it work on unseen data?

↓

GENERALIZATION Can it work in the real world?

LOOKING AHEAD

Where This Goes Next

This module is the first step on a longer path.

- Classical ML — you are here
- Statistics & Probability — next
- Model Optimization
- Feature Engineering
- Advanced ML
- Neural Networks
- Deep Learning

CLOSING

Machine learning is not about making computers "think." It is about using data to learn patterns that help computers make useful predictions and decisions.

Data → Patterns → Models → Predictions → Decisions

Now let's teach a computer something.